FinTrack — Frontend API Reference

> ***Purpose:*** *Complete API guide for the Angular frontend team.*
> ***Backend:*** *FinTrackCore API*
> ***Base URL (dev):*** `http://localhost:5027`
> ***Content-Type:*** `application/json`
> ***CORS origin (dev):*** `http://localhost:4200`

---

1. Universal response envelope

Every endpoint returns:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Optional message",
  "data": { },
  "meta": null
}
```

Paged list (`GET /api/Transactions`)

```
{
  "success": true,
  "statusCode": 200,
  "message": "",
  "data": [ ],
  "meta": {
    "totalData": 100,
    "totalPage": 5
  }
}
```

Mutation success (create / update / delete)

```
{
  "success": true,
  "statusCode": 201,
  "message": "Data saved successfully.",
  "data": { "id": 1 },
  "meta": null
}
```

Error

```
{
  "success": false,
```

```
  "statusCode": 400,
  "message": "Error message here",
  "data": null,
  "meta": null
}
```

HTTP status codes

Code	Meaning
200	OK
201	Created
400	Bad request / validation
401	Unauthorized (missing or expired token)
403	Forbidden (wrong user)
404	Not found
409	Conflict (duplicate, etc.)

Auth header (protected routes)

```
Authorization: Bearer {accessToken}
```

---

2. Auth flow (frontend must implement)

```
Register (POST /api/UserInfos) → redirect to login (no tokens)

Login / Google → store accessToken + refreshToken + user

Every API call → attach Bearer token

401 response → POST /api/Auths/refresh → retry request
Refresh fails → clear storage → redirect /login

Logout → POST /api/Auths/logout → clear storage
```

Do not poll refresh. Refresh only on 401 or shortly before expiry.

---

3. AuthController — `/api/Auths`

Method	Route	Auth	Description
POST	`/api/Auths/login`	No	Login with username/email + password
POST	`/api/Auths/google`	No	Google sign-in / sign-up

POST	`/api/Auths/refresh`	No	Refresh access token
POST	`/api/Auths/logout`	No	Revoke refresh token

POST `/api/Auths/login`

Request:

```
{
  "userNameOrEmail": "hridoy",
  "password": "Secret@123"
}
```

Response `data`:

```
{
  "accessToken": "eyJhbG...",
  "refreshToken": "abc123...",
  "expiresIn": 900,
  "user": {
    "id": 1,
    "userName": "hridoy",
    "email": "hridoy@example.com",
    "firstName": "Hridoy",
    "lastName": "Ahmed",
    "currencyCode": "BDT"
  }
}
```

POST `/api/Auths/google`

Request:

```
{
  "idToken": "GOOGLE_ID_TOKEN_FROM_GIS"
}
```

Response `data`: Same shape as login.

POST `/api/Auths/refresh`

Request:

```
{
  "refreshToken": "abc123..."
}
```

Response `data`: Same shape as login.

POST `/api/Auths/logout`

Request:

```
{
  "refreshToken": "abc123..."
}
```

Response:

```
{
  "success": true,
  "statusCode": 200,
  "message": "Logged out successfully.",
  "data": null,
  "meta": null
}
```

---

4. UserInfoController — `/api/UserInfos`

Method	Route	Auth	Description
POST	`/api/UserInfos`	No	Register new user
GET	`/api/UserInfos/{id}`	Yes	Get profile (own id only)
PUT	`/api/UserInfos/{id}`	Yes	Update profile (own id only)

POST `/api/UserInfos` — Register

Request:

```
{
  "userName": "hridoy",
  "email": "hridoy@example.com",
  "password": "Secret@123",
  "firstName": "Hridoy",
  "lastName": "Ahmed",
  "currencyCode": "BDT",
  "isActive": true
}
```

Response `data`:

```
{ "id": 1 }
```

Note: Register does not return tokens. Redirect user to login.

Side effects on register: Default COA (18 accounts) + current Financial Year are auto-created.

GET `/api/UserInfos/{id}`

Response `data`:

```
{
  "id": 1,
  "userName": "hridoy",
  "email": "hridoy@example.com",
  "firstName": "Hridoy",
  "lastName": "Ahmed",
  "currencyCode": "BDT",
  "isActive": true,
  "createdDate": "2026-08-25T09:46:30Z",
  "updatedDate": null
}
```

PUT `/api/UserInfos/{id}`

Request:

```
{
  "userName": "hridoy",
  "email": "hridoy@example.com",
  "password": "NewSecret@123",
  "firstName": "Hridoy",
  "lastName": "Ahmed",
  "currencyCode": "BDT",
  "isActive": true
}
```

`password` is optional — omit to keep current password.

Response `data`:

```
{ "id": 1 }
```

---

5. AccountTypeController — `/api/AccountTypes`

Method	Route	Auth	Description
GET	`/api/AccountTypes`	Yes	List all account types (lookup)

GET `/api/AccountTypes`

Response `data`:

```
[
  { "id": 1, "code": "ASSET",     "name": "Asset",     "normalBalance": "DEBIT"  },
  { "id": 2, "code": "LIABILITY", "name": "Liability", "normalBalance": "CREDIT" },
```

```
  { "id": 3, "code": "EQUITY",    "name": "Equity",    "normalBalance": "CREDIT" },
  { "id": 4, "code": "INCOME",    "name": "Income",    "normalBalance": "CREDIT" },
  { "id": 5, "code": "EXPENSE",   "name": "Expense",   "normalBalance": "DEBIT"  }
]
```

---

6. CoaController — `/api/Coas`

Method	Route	Auth	Description
GET	`/api/Coas`	Yes	List user's chart of accounts
GET	`/api/Coas/{id}`	Yes	Get one account
POST	`/api/Coas`	Yes	Create account
PUT	`/api/Coas/{id}`	Yes	Update account
DELETE	`/api/Coas/{id}`	Yes	Delete account

GET `/api/Coas`

Response `data`:

```
[
  {
    "id": 5,
    "userInfoId": 1,
    "parentId": 2,
    "accountTypeId": 1,
    "accountType": {
      "id": 1,
      "code": "ASSET",
      "name": "Asset",
      "normalBalance": "DEBIT"
    },
    "accountCode": "10100",
    "accountName": "Cash",
    "isSystemDefault": true,
    "isActive": true,
    "createdDate": "2026-08-25T09:46:30Z",
    "updatedDate": null
  }
]
```

Build COA tree in frontend using `parentId`.

GET `/api/Coas/{id}`

Response `data`: Single COA object (same shape as list item).

POST `/api/Coas`

Request:

```
{
  "parentId": 2,
  "accountTypeId": 1,
  "accountCode": "10400",
  "accountName": "Savings"
}
```

Response `data`:

```
{ "id": 20 }
```

PUT `/api/Coas/{id}`

Request:

```
{
  "parentId": 2,
  "accountName": "Petty Cash",
  "isActive": true
}
```

Response `data`:

```
{ "id": 5 }
```

DELETE `/api/Coas/{id}`

Response `data`:

```
{ "id": 5 }
```

System default accounts (`isSystemDefault: true`) cannot be deleted.

Default COA codes (seeded on register)

Code	Name	Type
10100	Cash	Asset
10200	Bank	Asset
10300	Mobile Wallet	Asset
20100	Credit Card	Liability
30100	Opening Balance	Equity
40100	Salary	Income

40200	Freelance Income	Income
50100	Food	Expense
50200	Transport	Expense
50300	Rent	Expense
50400	Utilities	Expense
50500	Shopping	Expense
50600	Entertainment	Expense

---

7. FinancialYearController — `/api/FinancialYears`

Method	Route	Auth	Description
GET	`/api/FinancialYears`	Yes	List visible years (max 2)
GET	`/api/FinancialYears/current`	Yes	Current active year
GET	`/api/FinancialYears/{id}`	Yes	Get by id

Auto-creates new year when calendar year changes. Returns current + previous year only (no future years).

GET `/api/FinancialYears`

Response `data`:


```
[
  {
    "id": 2,
    "userInfoId": 1,
    "year": 2026,
    "name": "FY 2026",
    "startDate": "2026-01-01T00:00:00Z",
    "endDate": "2026-12-31T00:00:00Z",
    "isActive": true,
    "isClosed": false,
    "createdDate": "2026-08-28T06:18:43Z",
    "updatedDate": null
  },
  {
    "id": 1,
    "userInfoId": 1,
    "year": 2025,
    "name": "FY 2025",
    "startDate": "2025-01-01T00:00:00Z",
    "endDate": "2025-12-31T00:00:00Z",
    "isActive": false,
    "isClosed": true,
    "createdDate": "2026-08-28T06:18:43Z",
```

```
    "updatedDate": "2026-08-28T06:18:43Z"
  }
]
```

GET `/api/FinancialYears/current`

Response `data` : Single financial year object (current calendar year).

GET `/api/FinancialYears/{id}`

Response `data` : Single financial year object.

---

8. TransactionTypeController — `/api/TransactionTypes`

Method	Route	Auth	Description
GET	`/api/TransactionTypes`	Yes	List all transaction types
GET	`/api/TransactionTypes/{id}`	Yes	Get by id

GET `/api/TransactionTypes`

Response `data` :

```
[
  { "id": 1, "code": "INCOME",          "name": "Income"          },
  { "id": 2, "code": "EXPENSE",         "name": "Expense"         },
  { "id": 3, "code": "TRANSFER",        "name": "Transfer"        },
  { "id": 4, "code": "OPENING_BALANCE", "name": "Opening Balance" }
]
```

---

9. TransactionController — `/api/Transactions`

Method	Route	Auth	Description
GET	`/api/Transactions`	Yes	Paged list
GET	`/api/Transactions/{id}`	Yes	Detail with voucher lines
POST	`/api/Transactions`	Yes	Create transaction

GET `/api/Transactions`

Query parameters:

Param	Type	Default	Description
`financialYearId`	long?	—	Filter by financial year
`transactionTypeId`	long?	—	Filter by type

`fromDate`	date?	—	Start date filter
`toDate`	date?	—	End date filter
`page`	int	1	Page number
`pageSize`	int	20	Items per page (max 100)

Example: `GET /api/Transactions?financialYearId=2&page=1&pageSize=20`

Response `data`:


```
[
  {
    "id": 1,
    "userInfoId": 1,
    "financialYearId": 2,
    "financialYear": { "id": 2, "year": 2026, "name": "FY 2026" },
    "transactionTypeId": 2,
    "transactionType": { "id": 2, "code": "EXPENSE", "name": "Expense" },
    "transactionDate": "2026-08-28T00:00:00Z",
    "amount": 500.00,
    "description": "Lunch",
    "createdDate": "2026-08-28T07:00:00Z",
    "updatedDate": null
  }
]
```


GET `/api/Transactions/{id}`

Response `data`:


```
{
  "id": 1,
  "userInfoId": 1,
  "financialYearId": 2,
  "financialYear": {
    "id": 2,
    "year": 2026,
    "name": "FY 2026",
    "startDate": "2026-01-01T00:00:00Z",
    "endDate": "2026-12-31T00:00:00Z",
    "isActive": true,
    "isClosed": false
  },
  "transactionTypeId": 2,
  "transactionType": { "id": 2, "code": "EXPENSE", "name": "Expense" },
  "transactionDate": "2026-08-28T00:00:00Z",
  "amount": 500.00,
  "description": "Lunch",
  "createdDate": "2026-08-28T07:00:00Z",
  "updatedDate": null,
```

```
  "voucherLines": [
    {
      "id": 1,
      "transactionId": 1,
      "coaId": 15,
      "coa": {
        "id": 15,
        "accountCode": "50100",
        "accountName": "Food",
        "accountType": { "id": 5, "code": "EXPENSE", "name": "Expense", "normalBalance":
"DEBIT" }
      },
      "lineNumber": 1,
      "debitAmount": 500.00,
      "creditAmount": 0.00,
      "createdDate": "2026-08-28T07:00:00Z"
    },
    {
      "id": 2,
      "transactionId": 1,
      "coaId": 5,
      "coa": {
        "id": 5,
        "accountCode": "10100",
        "accountName": "Cash",
        "accountType": { "id": 1, "code": "ASSET", "name": "Asset", "normalBalance": "DEBIT"
}
      },
      "lineNumber": 2,
      "debitAmount": 0.00,
      "creditAmount": 500.00,
      "createdDate": "2026-08-28T07:00:00Z"
    }
  ]
}
```


POST `/api/Transactions`

Request:


```
{
  "transactionTypeId": 2,
  "financialYearId": 2,
  "transactionDate": "2026-08-28",
  "amount": 500,
  "description": "Lunch",
  "debitCoaId": 15,
  "creditCoaId": 5
}
```


Response `data` :

```
{ "id": 1 }
```

Frontend UI → API mapping (transaction forms)

UI Action	transactionTypeId	debitCoaId	creditCoaId
Income (Salary → Bank)	1 INCOME	Asset (Bank/Cash)	Income (Salary)
Expense (Food ← Cash)	2 EXPENSE	Expense (Food)	Asset (Cash)
Transfer (Cash → Bank)	3 TRANSFER	Asset (Bank)	Asset (Cash)
Opening Balance	4 OPENING_BALANCE	Asset (Cash)	Equity (Opening Balance)

Example — Income ৳50,000 salary to bank:

```
{
  "transactionTypeId": 1,
  "financialYearId": 2,
  "transactionDate": "2026-08-28",
  "amount": 50000,
  "description": "August salary",
  "debitCoaId": 6,
  "creditCoaId": 12
}
```

(`debitCoaId` = Bank 10200, `creditCoaId` = Salary 40100)

---

10. ReportController — `/api/Reports`

Method	Route	Auth	Description
GET	`/api/Reports/dashboard`	Yes	Dashboard summary

GET `/api/Reports/dashboard`

Query parameters:

Param	Type	Description
`financialYearId`	long?	Optional — defaults to current year

Example: `GET /api/Reports/dashboard?financialYearId=2`

Response `data`:

```
{
  "financialYearId": 2,
  "financialYear": 2026,
  "totalBalance": 125000.00,
  "incomeThisMonth": 50000.00,
```

```
  "expenseThisMonth": 18500.00,
  "netThisMonth": 31500.00,
  "accountBalances": [
    {
      "coaId": 5,
      "accountCode": "10100",
      "accountName": "Cash",
      "accountTypeId": 1,
      "accountTypeCode": "ASSET",
      "balance": 5000.00
    },
    {
      "coaId": 6,
      "accountCode": "10200",
      "accountName": "Bank",
      "accountTypeId": 1,
      "accountTypeCode": "ASSET",
      "balance": 120000.00
    },
    {
      "coaId": 8,
      "accountCode": "20100",
      "accountName": "Credit Card",
      "accountTypeId": 2,
      "accountTypeCode": "LIABILITY",
      "balance": 0.00
    }
  ]
}
```


---

11. All endpoints — quick reference

#	Method	Route	Auth
1	POST	`/api/Auths/login`	No
2	POST	`/api/Auths/google`	No
3	POST	`/api/Auths/refresh`	No
4	POST	`/api/Auths/logout`	No
5	POST	`/api/UserInfos`	No
6	GET	`/api/UserInfos/{id}`	Yes
7	PUT	`/api/UserInfos/{id}`	Yes
8	GET	`/api/AccountTypes`	Yes
9	GET	`/api/Coas`	Yes
10	GET	`/api/Coas/{id}`	Yes

11	POST	`/api/Coas`	Yes
12	PUT	`/api/Coas/{id}`	Yes
13	DELETE	`/api/Coas/{id}`	Yes
14	GET	`/api/FinancialYears`	Yes
15	GET	`/api/FinancialYears/current`	Yes
16	GET	`/api/FinancialYears/{id}`	Yes
17	GET	`/api/TransactionTypes`	Yes
18	GET	`/api/TransactionTypes/{id}`	Yes
19	GET	`/api/Transactions`	Yes
20	GET	`/api/Transactions/{id}`	Yes
21	POST	`/api/Transactions`	Yes
22	GET	`/api/Reports/dashboard`	Yes

---

12. Angular frontend — suggested pages & APIs

Page	APIs used
Login	POST `/api/Auths/login`, POST `/api/Auths/google`
Register	POST `/api/UserInfos`
Dashboard	GET `/api/Reports/dashboard`, GET `/api/FinancialYears/current`
Profile	GET/PUT `/api/UserInfos/{id}`
Chart of Accounts	GET/POST/PUT/DELETE `/api/Coas`, GET `/api/AccountTypes`
Transactions list	GET `/api/Transactions`
Add Income	POST `/api/Transactions`, GET `/api/Coas`, GET `/api/TransactionTypes`
Add Expense	POST `/api/Transactions`, GET `/api/Coas`, GET `/api/TransactionTypes`
Transfer	POST `/api/Transactions`, GET `/api/Coas`
Transaction detail	GET `/api/Transactions/{id}`

---

13. Angular TypeScript interfaces

```typescript
export interface ApiResponse<T> {
  success: boolean;
  statusCode: number;
  message: string;
```

```ts
  data: T;
  meta?: { totalData: number; totalPage: number } | null;
}

export interface AuthTokenData {
  accessToken: string;
  refreshToken: string;
  expiresIn: number;
  user: AuthUser;
}

export interface AuthUser {
  id: number;
  userName: string;
  email: string;
  firstName: string;
  lastName?: string | null;
  currencyCode: string;
}

export interface UserInfo {
  id: number;
  userName: string;
  email: string;
  firstName: string;
  lastName?: string | null;
  currencyCode: string;
  isActive: boolean;
  createdDate: string;
  updatedDate?: string | null;
}

export interface AccountType {
  id: number;
  code: string;
  name: string;
  normalBalance: string;
}

export interface Coa {
  id: number;
  userInfoId: number;
  parentId?: number | null;
  accountTypeId: number;
  accountType?: AccountType;
  accountCode: string;
  accountName: string;
  isSystemDefault: boolean;
  isActive: boolean;
  createdDate: string;
  updatedDate?: string | null;
}
```

```ts
export interface FinancialYear {
  id: number;
  userInfoId: number;
  year: number;
  name: string;
  startDate: string;
  endDate: string;
  isActive: boolean;
  isClosed: boolean;
  createdDate: string;
  updatedDate?: string | null;
}

export interface TransactionType {
  id: number;
  code: string;
  name: string;
}

export interface Transaction {
  id: number;
  userInfoId: number;
  financialYearId: number;
  financialYear?: FinancialYear;
  transactionTypeId: number;
  transactionType?: TransactionType;
  transactionDate: string;
  amount: number;
  description?: string | null;
  createdDate: string;
  updatedDate?: string | null;
  voucherLines?: VoucherLine[];
}

export interface VoucherLine {
  id: number;
  transactionId: number;
  coaId: number;
  coa?: Coa;
  lineNumber: number;
  debitAmount: number;
  creditAmount: number;
  createdDate: string;
}

export interface DashboardData {
  financialYearId: number;
  financialYear: number;
  totalBalance: number;
  incomeThisMonth: number;
  expenseThisMonth: number;
```

```
  netThisMonth: number;
  accountBalances: AccountBalanceItem[];
}

export interface AccountBalanceItem {
  coaId: number;
  accountCode: string;
  accountName: string;
  accountTypeId: number;
  accountTypeCode: string;
  balance: number;
}

export interface CreateTransactionRequest {
  transactionTypeId: number;
  financialYearId: number;
  transactionDate: string;
  amount: number;
  description?: string;
  debitCoaId: number;
  creditCoaId: number;
}
```

---

14. Environment config (Angular)

```
// src/environments/environment.ts
export const environment = {
  production: false,
  apiBaseUrl: 'http://localhost:5027',
  googleClientId: 'YOUR_GOOGLE_OAUTH_CLIENT_ID.apps.googleusercontent.com'
};
```

---

15. Implementation order (frontend)

Phase	Tasks
1	Project setup, ApiService, Auth interceptor, token storage
2	Login, Register, Google auth
3	Main layout, profile page
4	Dashboard (Reports API)
5	COA tree page
6	Transaction list + create forms (Income / Expense / Transfer)

Document version: 1.0 — aligned with FinTrackCore API as of August 2026.